

AI Engineer — Internship Assignment

Context

You're joining a team that ships AI-first prototypes — LLM APIs, prompt engineering, RAG and multi-agent systems used to test product ideas quickly. This assignment mirrors a typical first task: take a fuzzy problem, scope it sensibly, and build a working POC.

The Problem

Build an **AI Mock Interview Coach** — a multi-agent system that runs a realistic mock interview for someone preparing for a target role and gives them useful feedback at the end.

The candidate provides:

- Target role (e.g., *Product Manager, Data Analyst, Frontend Engineer intern*)
- Optional: a 2–3 line background or resume snippet
- Focus area for this session (behavioral / technical / case / mixed)

Your system should:

1. **Conduct an interview** of roughly 5–7 turns with intelligent follow-ups (not just a fixed question list)
2. **Adapt** — probe deeper on weak answers, move on from strong ones, calibrate difficulty
3. **Evaluate** answers along multiple dimensions, not just "good/bad"
4. **Coach** — at the end, deliver structured feedback: strengths, gaps, specific things to practice

This is a small system, but a non-trivial one. The interesting parts are *designing the agents* and *making the prompts robust* to real conversational messiness.

Required Components

1. Multi-Agent Architecture

At least **3 distinct agents** with genuinely different roles. Suggested split (you may adapt):

Agents should genuinely differ in role and behaviour — not just three calls in a chain dressed up as "agents". Show us the orchestration logic that ties them together.

2. Prompt Engineering

Each agent needs a thoughtfully crafted system prompt. We'll look at:

- Role and scope clarity, persona consistency
- Structured outputs where useful (JSON for evaluator, markdown for coach, etc.)
- Handling of real-world messiness — vague answers, off-topic replies, partial correctness, "I don't know"

3. Interface

A simple CLI or notebook is fine. Streamlit / Gradio welcome but not required.

4. (Optional) Grounding

If you want answers grounded in real-world content (e.g., role-specific question banks, current interview trends), feel free to add web search or RAG — but it's **not required**. Don't build it unless it makes the prototype meaningfully better.

Deliverables

1. Code (GitHub repo or zip)

- Source code, modular and readable
- `requirements.txt`
- A `prompts/` folder with each agent's prompt as a separate file

2. README.md with:

- Setup and run instructions
- Architecture overview — what each agent does, how they're orchestrated
- Key design decisions and tradeoffs
- 2–3 example interview transcripts (one strong candidate, one weak, one tricky/edge case)